

IT happens @ RAALE

Containerized Audience Segmentation & Personalization Service

2 Hours 30 Minutes Build | 30 Minutes Report | Student Hackathon Problem Statement

Challenge Theme: Build a practical, deployable machine-learning service that understands OTT viewer behavior, groups users into meaningful behavioral segments, and exposes personalized recommendations through a containerized API.

THE BUSINESS NEED

OTT platforms serve large and diverse audiences. A single recommendation strategy cannot effectively serve viewers with different interests, engagement levels, session patterns, and genre preferences.

THE ENGINEERING GOAL

Convert raw user activity into an end-to-end product: data pipeline → clustering model → persisted model → API → evaluation service → reproducible Docker deployment.

1. PROBLEM STATEMENT

An OTT streaming platform generates a continuous stream of viewer activity: watch time, session duration, frequency of visits, preferred genres, completion behavior, and other engagement signals. Although this data contains valuable information about audience behavior, raw activity records are difficult to use directly for personalization decisions.

The platform therefore needs an automated audience intelligence service that can discover groups of users with similar behavior without relying on manually assigned labels. These groups should be useful for downstream personalization—for example, distinguishing highly engaged action viewers from occasional family-content viewers or short-session casual viewers.

Your challenge is to design and implement a complete, lightweight machine-learning service that learns these behavioral segments from the supplied tabular dataset and makes the result available through a REST API. The solution must be deployable through Docker Compose and must include an independent evaluation component that produces measurable evidence of model and system quality.

2. REAL-WORLD CONTEXT

Imagine an OTT company with hundreds of thousands or millions of users. The recommendation team cannot manually inspect every customer. Instead, the system needs to identify behavioral patterns automatically.

Viewer behavior	Possible interpretation
High watch time + long sessions	Highly engaged viewers who may respond to deeper content discovery.
Frequent short sessions	Users who may prefer quick, easily discoverable content.
Strong genre concentration	Users with focused content preferences.
Low activity + weekend usage	Occasional or low-frequency viewers who may need different engagement strategies.
Mixed genres + high activity	Exploratory viewers who may benefit from broader recommendations.

3. CHALLENGE OBJECTIVES

- Prepare and validate the supplied user-activity dataset for machine-learning use.
- Select meaningful behavioral features such as watch time, session duration, activity frequency, and genre preferences.
- Build an unsupervised clustering pipeline using a practical algorithm such as KMeans or Gaussian Mixture Model.
- Standardize numerical features so that variables with different scales do not dominate the clustering process.
- Determine a defensible number of clusters using appropriate evidence such as silhouette score, inertia/elbow analysis, or stability checks.
- Translate machine-generated cluster IDs into human-readable segment descriptions based on cluster characteristics.
- Persist the trained preprocessing and clustering pipeline so that the API does not retrain the model for every request.
- Expose the model through a REST endpoint that accepts a user profile and returns a segment plus recommendations.
- Package the complete solution as multiple Docker containers orchestrated by Docker Compose.
- Build an evaluator service that waits for the API to become healthy, tests representative inputs, calculates metrics, and writes metrics.json.
- Ensure that a clean machine can reproduce the system using a single documented startup command.
- Document methods, results, assumptions, limitations, and failed approaches honestly in REPORT.md.

4. EXPECTED SOLUTION

The expected solution is not just a notebook or a trained model. It is a small production-style service composed of independent components. Each component should have one clear responsibility and communicate through predictable interfaces and shared model/data volumes.

4.1 Recommended three-service architecture

Service	Responsibility	Output
trainer	Loads training data, validates features, scales data, trains the clustering model, creates segment metadata, and saves the complete inference pipeline.	Saved model under /models
api	Loads the persisted pipeline at startup, validates incoming JSON, assigns a segment, and produces recommendation results.	POST /recommend GET /health
evaluator	Waits for API health, submits test profiles, verifies responses, calculates clustering/evaluation metrics, and records results.	metrics.json

4.2 End-to-end flow

1. Dataset	→	2. Feature Engineering	→	3. Clustering
User activity records	→	Scale + encode behavioral features	→	KMeans / GMM + cluster analysis
3. Model	→	4. API	→	5. Evaluation
Persist pipeline in /models	→	POST /recommend	→	Metrics + metrics.json

5. DATA & FEATURE ENGINEERING

Use the provided tabular user activity dataset. Teams should inspect the schema rather than assuming that every column is useful. The objective is to create a compact behavioral representation of each user.

- Engagement features: total watch time, viewing frequency, number of sessions, or similar available measures.
- Session features: average session duration, typical session length, or related session statistics.
- Content-preference features: genre frequencies, top genres, genre diversity, or encoded genre indicators.
- Activity-pattern features: weekday/weekend behavior, recency, or other activity indicators if present.
- Data-quality handling: missing values, invalid numeric values, duplicate records, and unexpected categories should be handled explicitly.

Important constraint

Do not create artificial labels merely to make the clustering problem look supervised. The core task is unsupervised audience segmentation. If labels exist in the dataset, they may be used for additional analysis only when clearly documented; they must not secretly become the target used to train the clustering model.

6. MODEL DEVELOPMENT

A practical baseline is StandardScaler followed by KMeans. Teams may use another lightweight unsupervised approach such as Gaussian Mixture Model if they can justify the choice. The model should be deterministic through fixed random seeds wherever the chosen algorithm supports them.

- Establish a simple baseline before tuning.
- Compare a reasonable range of cluster counts rather than selecting K arbitrarily.
- Report silhouette score and at least one complementary indicator such as inertia/elbow behavior or stability.
- Inspect cluster sizes and feature summaries; a mathematically valid cluster can still be operationally unhelpful.
- Create segment names from observed cluster characteristics—for example, 'High-Engagement Action Viewers'—rather than assigning names randomly.
- Save preprocessing and model together so inference uses exactly the same transformations as training.

7. API CONTRACT

The service must expose a stable interface so that the evaluator can interact with it without knowing internal implementation details.

```
POST /recommend
{"user_id":"USR-8192", "watch_time_hours":32.5, "top_genres":["Action","Thriller"], "avg_session_mins":85.0}

Expected response
{"user_id":"USR-8192", "segment_id":3, "segment_name":"High-Engagement Action Viewers",
"recommendations":["Movie A","Movie B"], "distance_to_centroid":0.42}

GET /health
{"status":"ok", "model_loaded":true}
```

7.1 Input validation requirements

- Reject malformed JSON with an appropriate HTTP error.
- Validate required fields and numeric ranges.
- Handle missing or empty genre lists gracefully.
- Do not allow unexpected data types to crash the service.
- Return clear error messages without exposing internal stack traces.
- Keep the API responsive and avoid retraining during inference.

8. PERSONALIZATION LOGIC

The hackathon does not require a large recommendation engine. A transparent rule-based layer is sufficient. After the clustering model assigns a segment, map that segment to a small recommendation catalog derived from available genres or supplied content metadata.

Segment example	Recommendation behavior
High-Engagement Action Viewers	Prioritize action/thriller content with deeper viewing options.
Casual Short-Session Viewers	Prioritize short films, trending clips, or easily completed content.
Genre-Explorers	Mix preferred genres with controlled exploration of adjacent genres.
Low-Activity Viewers	Surface popular, low-friction content rather than assuming intensive preferences.

9. DOCKER & DEPLOYMENT REQUIREMENTS

- **docker-compose.yml**: must define trainer, api, and evaluator services.
- **Shared model volume**: trainer writes the persisted pipeline and API reads the same artifact.
- **Startup ordering**: evaluator must wait for API health rather than assuming that container start means application readiness.
- **Healthcheck**: API must expose a working /health endpoint suitable for Docker health monitoring.
- **Reproducibility**: pin dependency versions and random seeds where practical.
- **Container hygiene**: use a lean base image, avoid unnecessary packages, and run as a non-root user where feasible.
- **One-command startup**: after the documented prerequisites are installed, the expected path should be a single Docker Compose command.

10. EVALUATION & EVIDENCE

The evaluator is a first-class part of the solution. Teams must not only claim that the model works; they must produce evidence. The evaluation should cover both machine-learning quality and service correctness.

Area	Evidence expected
Clustering quality	Silhouette score and complementary cluster-quality evidence.
Cluster balance	Cluster counts/sizes and identification of extremely small or dominant clusters.
Stability	Consistent results across clean runs with fixed seeds and documented settings.
API correctness	Successful health check and valid recommendation responses for representative inputs.
Robustness	Graceful handling of invalid/missing fields and edge cases.
Reproducibility	Fresh Docker Compose run produces the expected artifacts without hidden manual steps.

11. REQUIRED OUTPUTS

- **docker-compose.yml** — orchestrates the three services.
- **trainer service** — trains and persists the clustering pipeline.
- **api service** — serves /recommend and /health.
- **evaluator service** — validates the service and generates evaluation results.
- **metrics.json** — machine-readable evaluation output.
- **REPORT.md** — methods, architecture, experiments, results, limitations, and process notes.
- **Dockerfiles / requirements** — sufficient for reproducible builds.
- **Sample API requests** — demonstrate successful inference.

12. REPORT.md — MINIMUM CONTENT

- Problem understanding and assumptions
- Dataset description and preprocessing
- Feature-selection rationale
- Model and hyperparameter choices
- How the number of clusters was selected
- Cluster profiles and segment naming logic
- API design
- Docker architecture
- Evaluation methodology and metrics
- Results and observations
- Failure cases / edge cases tested
- Limitations and future improvements
- Reproducibility instructions
- What the team tried and changed during development

13. EDGE CASES TO TEST

- Unknown or unseen genre value.
- Empty top_genres list.
- Zero watch time.
- Very large watch time or session duration.
- Missing required field.
- String supplied where a numeric value is expected.
- Negative values where they are not meaningful.
- Repeated API requests for the same profile.
- API request arriving before the model has loaded.
- Fresh deployment where the model volume does not yet contain a trained artifact.

14. WHAT NOT TO DO

- Do not build only a Jupyter notebook and call it the final solution.
- Do not hard-code one segment for every user.
- Do not retrain the model for every API request.

- Do not hide evaluation failures or manually edit metrics.json.
- Do not depend on a paid external API or internet service for core inference.
- Do not download a heavy language model when a lightweight clustering pipeline is sufficient.
- Do not claim production readiness without documenting limitations.
- Do not add complexity simply for appearance; every component should serve a measurable purpose.

15. CONSTRAINTS

- **Compute:** laptop/CPU-friendly solution expected; GPU is not required.
- **Model:** lightweight unsupervised clustering; KMeans or GMM is recommended.
- **External AI:** no OpenAI/Claude or other large language model is required inside the service.
- **External APIs:** no paid API key should be necessary.
- **Deployment:** Docker and Docker Compose.
- **Reproducibility:** fixed seeds and pinned dependencies where appropriate.
- **Time:** 2 hours 30 minutes for implementation and 30 minutes for the report.

16. MARKING SCHEME — 100 MARKS

Component	Marks
docker compose up works cleanly on first run with no hidden manual steps	15
Correct three-service architecture and model sharing through volume	10
Healthcheck and correct startup/readiness ordering	8
Container hygiene: pinned tags, non-root user, lean image size	7
API correctness, validation, sanitization, and error handling	10
Evaluator correctness and valid metrics.json generation	10
Segmentation quality and meaningful cluster interpretation	10
Reproducibility across clean runs	5
REPORT.md: methods, experiments, results, honesty, limitations, and future work	25
Total	100

17. SUGGESTED 2.5-HOUR EXECUTION PLAN

Time	Focus	Expected checkpoint
0–20 min	Understand dataset, define features, inspect data quality.	Feature plan finalized.
20–55 min	Build preprocessing + clustering trainer.	Model artifact generated.
55–90 min	Build API and recommendation mapping.	/health and /recommend work locally.
90–120 min	Create Dockerfiles, Compose, volumes, healthcheck.	All containers start correctly.
120–150 min	Build evaluator, metrics.json, edge tests.	Evidence generated automatically.
Next 30 min	REPORT.md, cleanup, reproducibility test, final demo.	Submission package ready.

18. FINAL SUCCESS CRITERIA

- A clean machine can start the complete system using the documented Docker Compose command.
- The trainer produces a real persisted clustering pipeline from the supplied data.
- The API loads the persisted model and does not retrain during inference.
- The API returns a segment and recommendation response for valid user profiles.
- The API exposes a meaningful health status.
- The evaluator independently verifies the service and generates metrics.json.
- The report explains both strengths and limitations using actual evidence.
- The complete solution is understandable enough for another developer to reproduce.

CORE PRINCIPLE

Build the smallest system that proves the complete idea. A good hackathon submission is not the one with the most code; it is the one that can be started, tested, evaluated, explained, and reproduced.